



SECURITY AUDIT OF
OP_NET Native Swap



Public Report

Version: 1.1

March 13, 2026

Verichains Lab

info@verichains.io

<https://www.verichains.io>

EXECUTIVE SUMMARY

This Security Audit Report was prepared by Verichains Lab on **March 13, 2026**. We would like to thank the OP_NET Team for trusting Verichains Lab in auditing smart contracts. Delivering high-quality audits is always our top priority.

This audit focused on identifying security flaws in code and the design of the OP_NET Native Swap smart contract. The scope of the audit is limited to the source code files provided to Verichains. Verichains Lab completed the assessment using manual, static, and dynamic analysis techniques.

During the audit and penetration testing process, the audit team had identified some issues in the source code.

TABLE OF CONTENTS

1. MANAGEMENT SUMMARY	4
1.1. About Native Swap	4
1.2. Audit Methodology	4
1.3. Disclaimer	4
1.4. Acceptance Minute	5
2. AUDIT OVERVIEW	6
2.1. Audit Scope	6
2.2. Overview	6
3. FINDINGS	9
3.1. C-01 - resetProviderOnDust causes reportUTXOUsed fails to prevent double spending [CRITICAL] [FIXED]	9
3.2. C-02 - Incorrect AMM mechanism [CRITICAL] [FIXED]	12
3.3. H-01 - Wrong cancel withdraw penalty [HIGH] [FIXED]	14
3.4. M-01 - Revert in swap cause user to lose their btc without receiving tokens [MEDIUM] [ACKED] .	16
3.5. M-02 - ABI Decode Mismatch in reserve function [MEDIUM] [FIXED]	18
3.6. L-01 - ensureNotContract should not place inside ensureNotPaused [LOW] [FIXED]	19
3.7. I-01 - Swap mechanism risk [INFO] [ACKED]	20
3.8. I-02 - Price is updated multiple times in a block [INFO] [ACKED]	21
4. VERSION HISTORY	24

1. MANAGEMENT SUMMARY

1.1. About Native Swap

NativeSwap is an automated market maker (AMM) that operates directly on Bitcoin Layer 1 without taking custody of BTC. Built on OPNet's consensus layer, it enables trustless token swaps by tracking the cumulative effect of trades through virtual accounting while actual Bitcoin moves directly between traders.

1.2. Audit Methodology

Our security audit process for smart contract includes two steps:

- Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using public and our in-house smart contract security analysis tool.
- Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that were considered during the audit of the smart contract:

- Integer Overflow and Underflow
- Timestamp Dependence
- Race Conditions
- Transaction-Ordering Dependence
- DoS with (Unexpected) revert
- DoS with Block Gas Limit
- Gas Usage, Gas Limit and Loops
- Redundant fallback function
- Unsafe type Inference
- Reentrancy
- Explicit visibility of functions state variables (external, internal, private and public)
- Logic Flaws

For vulnerabilities, we categorize the findings into categories as listed in table below, depending on their severity level:

Severity Level	Description
CRITICAL	A vulnerability that can disrupt the contract functioning; creates a critical risk to the contract; required to be fixed immediately.
HIGH	A vulnerability that could affect the desired outcome of executing the contract with high impact; needs to be fixed with high priority.
MEDIUM	A vulnerability that could affect the desired outcome of executing the contract with medium impact in a specific scenario; needs to be fixed.
LOW	An issue that does not have a significant impact, can be considered as less important.

1.3. Disclaimer

OP_NET Team acknowledges that the security services provided by Verichains, are conducted to the best of their professional abilities but cannot guarantee 100% coverage of all security vulnerabilities. OP_NET Team understands and accepts that despite rigorous auditing, certain vulnerabilities may remain

undetected. Therefore, OP_NET Team agrees that Verichains shall not be held responsible or liable, and shall not be charged for any hacking incidents that occur due to security vulnerabilities not identified during the audit process.

1.4. Acceptance Minute

This final report served by Verichains to the OP_NET Team will be considered an Acceptance Minute. Within **seven** days, if no any further responses or reports is received from the OP_NET Team, the final report will be considered fully accepted by the OP_NET Team without the signature.

2. AUDIT OVERVIEW

2.1. Audit Scope

The audit focus on identifying security flaws in the code of OP_NET Native Swap smart contract. It was first conducted on commit 00022e788fb6dfd59ad38a16e31f636c7cc88560 from git repository <https://github.com/btc-vision/native-swap>.

This table summarizes the report revision history, mapping each report version to its corresponding Git commit hash.

Report Version	Git Commit
0.1	00022e788fb6dfd59ad38a16e31f636c7cc88560
0.2	5175d8688e6e3db85324ab92a7427247b0690fbb
0.3	7215a845984809fbd6b7f1b49ea90270d7b8b5ce
0.4	7215a845984809fbd6b7f1b49ea90270d7b8b5ce
1.0	5260bf0658d0948da3cfa40374f61804886aa6d3
1.1	5260bf0658d0948da3cfa40374f61804886aa6d3

2.2. Overview

NativeSwap is a non-custodial system that allows users to buy OP_NET tokens directly with Layer-1 BTC, eliminating the need to bridge Bitcoin to OP_NET. It maintains an **internal price oracle** and a queue-based execution system where sellers list their tokens to be sold for BTC. For standard pools, it uses the constant product formula ($k = T \times B$) for internal price discovery; sell pressure pushes the price down and buy pressure pushes the price up. For stable pairs (e.g., stablecoins pegged to the same asset), NativeSwap also supports a **StableSwap pool based on Curve's invariant**, which provides lower slippage for assets expected to trade near parity. The result is a DEX-like experience that respects Bitcoin's irreversible settlement rules while keeping sellers and buyers in control of their native assets at all times.

2.2.1. Key Capabilities

- Trustless pool creation** - Token issuers seed a pool by defining an initial floor price, uploading an initial token inventory, and optionally enabling anti-bot guardrails (a temporary per-reservation cap during the first blocks plus a lasting percentage cap on how much inventory can be reserved). Only the token deployer can initialize a pool, ensuring the quote is set by the asset owner.
- Deterministic seller queues** - Sellers list tokens by depositing them into NativeSwap and attaching a CSV P2WSH payout script. Once listed, sellers must wait for their tokens to be sold; they cannot cancel their listing unless the protocol's withdraw mode is activated. Listings can choose normal or priority placement; priority sellers pay an additional BTC fee and a small token tax but jump to the front of the queue.
- Reservation-based trading** - Buyers lock a quote by creating a reservation that specifies their satoshi budget and the minimum tokens they expect. NativeSwap immediately matches tokens across sellers in the queue and records the exact quote in effect for that block.
- Atomic Layer-1 settlement** - During swap execution, the protocol automatically verifies the Bitcoin transaction outputs against the reservation. Tokens are distributed based on the amount the sellers' payout scripts received, ensuring trustless delivery in a single atomic step.

5. **Dynamic fee sharing** - Completed trades pay a flexible fee (driven by trade size, utilization, and recent volatility) that is collected in tokens and forwarded to the staking contract, creating a native reward loop for the broader ecosystem.
6. **Graceful unwind mechanics** - Because listings are otherwise locked, the protocol includes an emergency “withdraw mode” that can only be triggered by the contract deployer. This serves as a global emergency exit, allowing all sellers to retrieve their unsold inventory if the protocol needs to be sunset or paused indefinitely.

2.2.2. Pool and Trade Lifecycle

1. **Pool launch** - The token owner defines the initial quote, contributes the first tranche of tokens, configures anti-bot settings, and designates a Bitcoin receiver that is compatible with the protocol’s CSV requirements. This initial deposit becomes the canonical reference for price discovery.
2. **Ongoing listings** - Additional sellers submit tokens plus a payout script. NativeSwap verifies the receiver, charges the priority tax if applicable, and places the seller into the normal or priority queue. Virtual reserves are updated to reflect the economic effect of newly listed inventory.
3. **Reservation phase** - Any EOA can reserve tokens by paying the base reservation fee in BTC, defining their satoshi budget, minimum acceptable tokens, and an optional activation delay (up to three blocks). The system enforces pool-specific caps so a single reservation cannot monopolize more than the configured percentage of total token inventory.
4. **Swap execution** - The buyer constructs a single Bitcoin transaction that sends BTC to the sellers’ payout scripts and simultaneously triggers the OP_NET swap. During execution, the protocol inspects the Bitcoin outputs and matches them to the reserved sellers. Tokens are distributed to the buyer based on the actual amount of satoshis each seller received (allowing for partial fills if the payment differs from the reservation), after which the protocol applies dynamic fees and updates the virtual reserves.
5. **Cleanup and purging** - The purge queue maintains contract state cleanliness by handling expired reservations and sellers with dust amounts. Reservations enter the purge queue when their execution window expires, when a seller needs to be reset due to dust amounts below minimum reservation thresholds, or when a swap is attempted after expiry. The purge process is automatically triggered before any new reservation creation or token listing. Each purge operation processes batches of sellers before yielding, restoring their reserved target tokens and marking them as purged to prevent double-processing. This keeps queues fresh and prevents state bloat even if users abandon their reservations.

2.2.3. Trader Experience

1. **Guaranteed quote window** - Reservations snapshot the exact quote for the block in which they were created. As long as the buyer broadcasts BTC before the reservation expires, the swap executes at that stored price, shielding traders from sandwich attacks and late price swings.
2. **Activation delay** - Buyers can request a delay (0-3 blocks) before their reservation becomes consumable, giving them time to coordinate multisig signatures or hardware wallet approvals without losing their spot.
3. **Minimum trade size and anti-bot caps** - Pools enforce both global minimum trade sizes (10000 sats) and configurable per-reservation ceilings. This ensures every reservation represents meaningful flow and prevents automated sniping at launch.
4. **Layer-1 proof of payment** - The protocol reads the Bitcoin transaction directly, so there is no need to trust relayers or bridges. If a buyer underpays, only the delivered portion is fulfilled; if they overpay, the seller keeps the extra BTC, creating strong incentives to send the exact quoted amount.

2.2.4. Economic Design

1. **Fee structure** - Two BTC-denominated platform fees exist: the reservation fee charged to buyers and the priority fee charged to sellers seeking expedited queue placement. Both are capped to prevent abusive settings and are fully enforced by checking Bitcoin outputs. Trading fees are charged in tokens and adapt to market conditions through a logarithmic formula that considers trade size, recent volatility, and current pool utilization.
2. **Anti-gaming mechanisms** - Reservation caps, purge queues, and dust burn rules eliminate profit from grieving or dust spam. Sellers who attempt to relist immediately after being partially filled must wait until their outstanding reservations are cleared, preventing double counting of the same inventory.

2.2.5. Operational Controls and Safety

1. **EOA-only enforcement** - All interactive actions require the caller to be an externally owned account; contracts cannot interact with NativeSwap when it is live. Combined with a built-in re-entrancy guard, this blocks automated exploits that depend on contract callbacks.
2. **Emergency Exit (Withdraw Mode)** - The contract deployer can trigger a one-way “withdraw mode” in extreme cases. This disables all new reservations and listings protocol-wide, serving as an emergency exit that lets all sellers retrieve their listed tokens immediately. Once activated, withdraw mode cannot be reverted, ensuring a permanent and safe shutdown path.
3. **Timeout discipline** - Users who let reservations expire are temporarily timed out and cannot create a new one until a cooldown window elapses. This throttles abusive reservation spam and keeps the seller queues flowing.
4. **Continuous monitoring** - The protocol emits detailed events for interactive actions (listing, reservation creation, swap execution, cancellations, withdrawals). Metrics like virtual reserves, queue sizes, and fee settings are available through read functions rather than event streams, so indexers should combine emitted events with state reads to warn users if a pool approaches risk limits such as maximum allowable price impact.

NativeSwap combines these components into a cohesive system where Bitcoin-native traders enjoy predictable, atomic execution, and OP_NET token sellers can sell their holdings with fine-grained control over their exit price and priority. The mix of internal price discovery math, queue-based execution, and reservation locks keeps the user experience familiar while staying faithful to the constraints of the UTXO world.

3. FINDINGS

3.1. C-01 - `resetProviderOnDust` causes `reportUTXOUsed` fails to prevent double spending [CRITICAL] FIXED

SCOPE

- Commit: 00022e788fb6dfd59ad38a16e31f636c7cc88560
- Files:
 - `src/managers/TradeManager.ts`
 - `src/models/Provider.ts`

DESCRIPTION

When executing a swap with `executeNormalOrPriorityTrade`, the contract calls `reportUTXOUsed` to prevent an UTXO from double spending (one UTXO use for two reservations with same `BtcReceiver`).

```
// src/managers/TradeManager.ts#L159-L213
private executeNormalOrPriorityTrade(
  provider: Provider,
  requestedTokens: u128,
  satoshisSent: u64,
): void {
  const actualTokens: u128 = this.getTargetTokens(
    satoshisSent,
    requestedTokens,
    provider.getLiquidityAmount(),
  );
  const actualTokens256: u256 = actualTokens.toU256();
  if (!actualTokens.isZero()) {
    this.ensureReservedAmountIsValid(provider, requestedTokens);
    this.ensureProviderHasEnoughLiquidity(provider, actualTokens);
    const actualTokensSatoshis: u64 = tokensToSatoshis(
      actualTokens256,
      this.quoteAtReservation,
    );
    provider.subtractFromReservedAmount(requestedTokens);
    if (
      !provider.isLiquidityProvisionAllowed() &&
      provider.getQueueIndex() !== INITIAL_LIQUIDITY_PROVIDER_INDEX
    ) {
      provider.allowLiquidityProvision();
      const totalLiquidity: u128 = provider.getLiquidityAmount();
      // floor(totalLiquidity / 2)
      const halfFloor: u128 = SafeMath.div128(totalLiquidity, u128.fromU32(2));
      // CEIL = floor + (totalLiquidity & 1)
      const halfCred: u128 = u128.add(
        halfFloor,
        u128.and(totalLiquidity, u128.One), // adds 1 iff odd
      );
      // track that we virtually added those tokens to the pool
      this.liquidityQueueReserve.addToTotalTokensSellActivated(halfCred.toU256());
      this.emitActivateProviderEvent(provider);
    }
    provider.subtractFromLiquidityAmount(actualTokens);
    this.resetProviderOnDust(provider);
    this.increaseTokenReserved(requestedTokens);
    this.increaseTotalTokensPurchased(actualTokens256);
    this.increaseSatoshisSpent(actualTokensSatoshis);
    this.reportUTXOUsed(provider.getBtcReceiver(), actualTokensSatoshis);
  } else {
    this.restoreReservedLiquidityForProvider(provider, requestedTokens);
  }
}
```

In a swap, if there are two providers sharing the same `BtcReceiver`, the first provider will be reset when only dust remains, as determined by `resetProviderOnDust`.

```
// src/managers/TradeManager.ts#L260-L269
private resetProviderOnDust(provider: Provider): void {
    const satoshis = tokensToSatoshis128(
        provider.getLiquidityAmount(),
        this.quoteAtReservation,
    );
    if (satoshis < STRICT_MINIMUM_PROVIDER_RESERVATION_AMOUNT_IN_SAT) {
        this.providerManager.resetProvider(provider, false, false);
    }
}
```

Resetting a provider will set its `BtcReceiver` to empty.

```
// src/models/Provider.ts#L420-L423
public resetListingProviderValues(): void {
    this.providerData.resetListingProviderValues();
    this.setBtcReceiver('');
}
```

As a result, all subsequent `provider.getBtcReceiver()` calls will return an empty string, and the consumed mark will be associated with the empty address rather than the first provider's `BtcReceiver`.

```
// src/managers/TradeManager.ts#L90-L100
protected reportUTXOUsed(address: string, value: u64): void {
    const consumedAlready = this.consumedOutputsFromUTXOs.has(address)
        ? this.consumedOutputsFromUTXOs.get(address)
        : 0;
    if (consumedAlready === 0) {
        this.consumedOutputsFromUTXOs.set(address, value);
    } else {
        this.consumedOutputsFromUTXOs.set(address, SafeMath.add64(value, consumedAlready));
    }
}
```

Therefore, when processing other providers with the same `BtcReceiver`, `getSatoshisSent` will continue counting already consumed UTXOs, since `consumedOutputsFromUTXOs` was only set for the empty address.

```
// src/managers/TradeManager.ts#L102-L123
protected getSatoshisSent(address: string): u64 {
    let totalSatoshis: u64 = 0;
    const outputs = Blockchain.tx.outputs;
    for (let i = 0; i < outputs.length; i++) {
        const output = outputs[i];
        if (output.to === address) {
            totalSatoshis = SafeMath.add64(totalSatoshis, output.value);
        }
    }
    const consumedSatoshis: u64 = this.consumedOutputsFromUTXOs.has(address)
        ? this.consumedOutputsFromUTXOs.get(address)
        : 0;
    if (totalSatoshis < consumedSatoshis) {
        throw new Revert('Impossible state: Double spend detected.');
```

This will cause double spending and the buyer will get more tokens while only pay for only one reservation.

UPDATES

This issue was fixed in commit [fab7913bb0092cf8afd654144070303cd3456ca2](#) .

3.2. C-02 - Incorrect AMM mechanism [CRITICAL] FIXED

SCOPE

- Commit: 7215a845984809fbd6b7f1b49ea90270d7b8b5ce
- Files:
 - `src/operations/ListTokensForSaleOperation.ts`
 - `src/managers/TradeManager.ts`

DESCRIPTION

In a classic constant-product Automated Market Maker:

- **k remains constant during normal swaps.** When someone performs a swap, the pool rebalances the reserves, but the product of the two reserves ($k = x \times y$) stays exactly the same (new k can be higher if fee collected).
- **k increases when liquidity is added** (and decreases when liquidity is removed).
When a liquidity provider adds tokens proportionally, the pool grows, the new k becomes larger than before, and this deeper liquidity reduces price impact/volatility for future swaps.

In the current code of NativeSwap:

- **When someone list liquidity**, it is activated in two halves: the first half is activated immediately, while the second half is activated only when a swap occurs.
The previous version of NativeSwap added these tokens directly to `virtualTokenReserve` , which effectively increased liquidity (and thus increased the k constant). The current version, however, adds them to `totalTokensSellActivated` instead.

```
// src/managers/TradeManager.ts#L197
this.liquidityQueue.increaseTotalTokensSellActivated(deltaHalf.toU256());
// src/managers/LiquidityQueueReserve.ts#L107
this.liquidityQueueReserve.addToTotalTokensSellActivated(halfCred.toU256());
```

When `updateVirtualPoolIfNeeded` is called, the amount in `totalTokensSellActivated` is added to `virtualTokenReserve` , while `virtualSatoshisReserve` is simultaneously decreased by a corresponding amount to keep the k constant unchanged. It now behaves like a sell transaction (sending tokens into the pool and taking the satoshis out), without increasing the overall k .

This means actual pool liquidity does not increase when someone lists liquidity. If this is the protocol's choice, we must change the BTC contribution removal logic because we have never added any liquidity yet. Instead, the contract should now reduce virtual token and increase virtual satoshi back (simulating a buy transaction) when canceling liquidity. Otherwise, we must bring back the logic of adding liquidity.

Another problem is that when swapping, k is also decreased because of the rounding down when recalculating virtual satoshi and the fee deducted directly from virtual token reserve (no recalculation of virtual satoshi). This means the actual k is decrease for every swap.

```
// src/managers/LiquidityQueue.ts#L529
T = SafeMath.add(T, dT_add);
B = SafeMath.div(initialK, T);

// src/managers/LiquidityQueue.ts#L563
T = SafeMath.sub(T, dT_buy);
// Calculate B to maintain constant product
B = SafeMath.div(k, T);

// src/managers/LiquidityQueue.ts#341
```

```
public distributeFee(totalFee: u256): void {  
    this.decreaseVirtualTokenReserve(totalFee);  
    this.decreaseTotalReserve(totalFee);  
    addAmountToStakingContract(totalFee);  
}
```

RECOMMENDATION

If the current version of `NativeSwap` stays without the add/remove liquidity feature, we need to make some changes to keep k constant:

- For the fee, we can store it and subtract it in `updateVirtualPoolIfNeeded` to recalculate B and keep k constant.
- When recalculating B , we can round it up and allow B (and k) to slightly increase.
- For cancel listing, we can remove the BTC contribution logic and reverse the list liquidity logic by increasing virtual satoshi and decreasing virtual token when `updateVirtualPoolIfNeeded` is called.
- The slashing logic now only needs to work on the token distribution; there's no need to adjust the virtual reserves anymore. This logic should be reconsidered where the slashed tokens should go and how can it incentive user to list liquidity.

UPDATES

This issue was fixed

3.3. H-01 - Wrong cancel withdraw penalty [HIGH] FIXED

SCOPE

- Commit: 7215a845984809fbd6b7f1b49ea90270d7b8b5ce
- Files:
 - `src/operations/CancelListingOperation.ts`
 - `src/managers/LiquidityQueue.ts`

DESCRIPTION

Base on the document:

The slashing mechanism is essential for preventing queue manipulation and enabling the Queue Impact feature. Without slashing, malicious actors could artificially inflate the queue to manipulate prices, then cancel without consequence. When a seller cancels their queued order, they face a penalty based on how long their order has been in the queue: * Grace Period ($\delta < \delta_0$): 50% penalty - sellers lose half their tokens immediately * Ramp Period ($\delta \geq \delta_0$): Penalty increases linearly from 50% to 90% over 14 days

The slashed tokens are handled as follows:

- 50% (the base penalty) is returned to the virtual token reserve, benefiting all participants
- Additional penalty amounts are sent to the staking contract (since LPs are not yet implemented)

Current implementation of cancel withdraw penalty is not follow the documentation. For example, let say that the penalty is 70%:

- 30% tokens will be refunded to the user `this.transferLiquidityBack(refundAmount);`
- `accruePenalty` transfer all 70% penalty to staking contract `addAmountToStakingContract(penaltyU256);`
- `accruePenalty` decrease both virtual token and virtual satoshi by $70 - 50 = 20\%$ (`penaltyLeft`) so the pool liquidity get decrease by 20%

```
// src/operations/CancelListingOperation.ts#L25-L38
public override execute(): void {
  const listedAtBlock: u64 = this.provider.getListedTokenAtBlock();
  this.checkPreConditions(listedAtBlock);

  const initialAmount: u128 = this.provider.getLiquidityAmount();
  const penaltyAmount: u128 = this.calculatePenalty(listedAtBlock, initialAmount);
  const halfToCharge: u128 = this.calculateHalfToCharge(initialAmount, penaltyAmount);
  const refundAmount: u256 = this.calculateRefund(initialAmount, penaltyAmount);

  this.prepareProviderForRefund();
  this.transferLiquidityBack(refundAmount);
  this.postProcessQueues(refundAmount, penaltyAmount, halfToCharge);
  this.emitListingCanceledEvent(initialAmount, penaltyAmount);
}
```

```
// src/managers/LiquidityQueue.ts#L189-L225
public accruePenalty(penalty: u128, half: u128): void {
  if (!penalty.isZero()) {
    this.ensurePenaltyNotLessThanHalf(penalty, half);

    const penaltyU256: u256 = penalty.toU256();
    const penaltyLeft = SafeMath.sub128(penalty, half);
    const penaltyLeftU256: u256 = penaltyLeft.toU256();
  }
}
```

```
this.decreaseTotalReserve(penaltyU256);
if (!penaltyLeftU256.isZero()) {
  // Get current state BEFORE modifications
  const currentBTC = u256.fromU64(this.virtualSatoshisReserve);
  const currentTokens = this.virtualTokenReserve;
  if (!currentBTC.isZero() && !currentTokens.isZero()) {
    // Calculate BTC to remove BEFORE modifying tokens
    const btcToRemove = SafeMath.div(
      SafeMath.mul(penaltyLeftU256, currentBTC),
      currentTokens,
    );
    // Now remove BOTH
    this.decreaseVirtualTokenReserve(penaltyLeftU256);
    if (
      !btcToRemove.isZero() &&
      btcToRemove.toU64() <= this.virtualSatoshisReserve
    ) {
      this.decreaseVirtualSatoshisReserve(btcToRemove.toU64());
    }
  }
  addAmountToStakingContract(penaltyU256);
}
```

UPDATES

This issue was fixed. NativeSwap is currently disable normal cancel operation.

3.4. M-01 - Revert in swap cause user to lose their btc without receiving tokens

[MEDIUM] ACKED

SCOPE

- Commit: 5175d8688e6e3db85324ab92a7427247b0690fbb
- Files:
 - src/contracts/NativeSwap.ts

DESCRIPTION

To execute a swap, the buyer must send BTC to the seller through the swap execution UTXO. The problem is that BTC transactions cannot be reverted by the OP_NET chain. Therefore, if the swap function on OP_NET reverts for any reason, the buyer will lose their BTC without receiving tokens on OP_NET.

```
// src/contracts/NativeSwap.ts#L575-L585
private swap(calldata: Calldata): BytesWriter {
  this.ensureNotPaused();
  this.ensureWithdrawModeNotActive();
  const token: Address = calldata.readAddress();
  this._tokenAddress = token.clone();
  this._swap(token);
  return new BytesWriter(0);
}
```

For example, suppose a user executes a normal swap on the Bitcoin chain and waits for the transaction to be included in a block. Meanwhile, the NativeSwap admin notices some abnormal transactions and decides to pause NativeSwap immediately for investigation. They send a pause transaction with a very high gas fee to ensure it is prioritized. If this pause transaction is confirmed before the swap transaction, the swap execution will revert, causing the buyer to lose the BTC they already sent to the seller. Later, the reservation will be purged and the tokens returned to the queue once the admin resumes operations after the investigation.

To mitigate this, the validation of BTC sent should be handled in a separate function (triggered with Blockchain.call) that cannot be reverted under any condition. This function should:

- Validate the BTC sent,
- Mark the transaction as processed,
- Mark the reservation as swapped, even if the protocol is paused.

All token transfers and pool state calculations can then be retrigged once the protocol is resumed.

This ensures that buyers will not lose their BTC without receiving tokens, while still allowing malicious or abnormal transactions to be paused safely.

UPDATES

This finding has been acknowledged by the team.

The entire smart contract and frontend have been designed to make this highly unlikely to happen and to do everything possible to prevent people from losing Bitcoin through various mechanisms, such as:

- Mempool watcher for RBF of transactions on the frontend during the swap call
- Reservation process
- Swap call designed to not revert under any possible valid condition (most checks happen on the reserve side)
- Swap expiration fallback mechanism to handle cases where an expired reservation is still mined
- CSV 1 addresses, reorg protection, and transaction pinning protection

After extensive testing on testnet, including stress tests at extremely high levels of transaction much greater than Bitcoin Mainnet's capabilities, network congestion, and many other behavior tests with wallet protection, CSV checks, and mempool policy checks, everything has proven robust and safe.

You can check a high-level explanation here: <https://x.com/bc1anakun/status/1973281422023475302>

The one point I do agree on is the pause contract example. This is a bug we will fix by still allowing swaps when the contract is paused, but by blocking reservations and listings. This way, any pending swaps will work, but no new ones can be initialized.

3.5. M-02 - ABI Decode Mismatch in `reserve` function [MEDIUM] FIXED

SCOPE

- Commit: 7215a845984809fbd6b7f1b49ea90270d7b8b5ce
- Files:
 - `src/contracts/NativeSwap.ts`

DESCRIPTION

The `forLP bool` parameter was removed from the `reserve` function in a previous version, but the ABI has not been updated yet, causing a mismatch during calls.

If a user calls the function using the old ABI with the `forLP` parameter, the contract reads `activationDelay` immediately after `minimumAmountOut`, effectively read the leftover `forLP` bytes incorrectly. This results in `activationDelay` becoming unpredictable and the actual activation delay being ignored.

```
// src/contracts/NativeSwap.ts#L119-L122
public override execute(method: Selector, calldata: Calldata): BytesWriter {
  switch (method) {
    case encodeSelector('reserve(address,uint64,uint256,bool,uint8)': // redundant bool here
      return this.reserve(calldata);
```

```
// src/contracts/NativeSwap.ts#L461-L474
private reserve(calldata: Calldata): BytesWriter {
  this.ensureNotPaused();
  this.ensureWithdrawModeNotActive();

  const token: Address = calldata.readAddress();
  const maximumAmountIn: u64 = calldata.readU64();
  const minimumAmountOut: u256 = calldata.readU256();
  const activationDelay: u8 = calldata.readU8();
  this._tokenAddress = token.clone();

  this._reserve(token, maximumAmountIn, minimumAmountOut, activationDelay);

  return new BytesWriter(0);
}
```

UPDATES

This issue was fixed

3.6. L-01 - `ensureNotContract` should not place inside `ensureNotPaused` [LOW] **FIXED**

SCOPE

- Commit: 7215a845984809fbd6b7f1b49ea90270d7b8b5ce
- Files:
 - `src/contracts/NativeSwap.ts`

DESCRIPTION

In `NativeSwap`, there is an `ensureNotContract` guard designed to prevent contracts from calling and exploiting the function, but it is currently placed inside `ensureNotPaused`.

```
// src/contracts/NativeSwap.ts#L870-L876
private ensureNotPaused(): void {
  if (this._isPaused.value) {
    throw new Revert(`NATIVE_SWAP: Contract is currently paused. Try again later.`);
  }
  this.ensureNotContract();
}
```

The `ensureNotPaused` guard is unrelated to whether the executor is a contract or not, so `ensureNotContract` should not be placed inside `ensureNotPaused` but in entry point functions which intend for user call only.

This could lead to issues in the future. For example, if a new feature or change allows a function to be executed while the contract is paused, a developer might remove `ensureNotPaused`, which would also mistakenly remove the contract guard. This would create a security vulnerability and open the system to potential attacks.

UPDATES

This issue was fixed.

3.7. I-01 - Swap mechanism risk [INFO] ACKED

SCOPE

- Commit: 5175d8688e6e3db85324ab92a7427247b0690fbb
- Files:
 - src/managers/TradeManager.ts
 - src/models/Reservation.ts

DESCRIPTION

When a reservation expires, it enters a fallback mode where the current quote is used instead of the reserved quote. However, there's no validation to ensure that the buyer still receives at least the minimum amount they requested. This effectively bypasses all slippage protection, and the user might end up receiving significantly less than the minimum amount shown during the reservation.

```
// Reservation.ts
public isExpired(): boolean {
    return Blockchain.block.number > this.reservationData.expirationBlock;
}

// TradeManager.ts - executeTradeExpired uses current quote without checking minimum
this.quoteToUse = currentQuote;
```

Moreover, the `executeExpired` function can be risky. For example, a user might attempt to call `executeExpired` when they notice the reservation has expired but hasn't been purged yet. However, by the time the Bitcoin transaction is mined and executed on OP_NET, the reservation might already have been purged. In this case, the OP_NET transaction would revert, and the user would not receive any tokens even though their BTC had already been sent to the seller.

The `executeExpired` function should not be encouraged for users to call; it should only serve as a fallback mechanism when the swap transaction arrives late to OP_NET.

With the current approach, there is always a risk that the Bitcoin transaction is confirmed too late - after the reservation has already been purged, the buyer lost their BTC while the seller receive the profit.

UPDATES

This finding has been acknowledged by the OP_NET team:

[this finding] is not a bug. This check is done in the reserve phase. You cannot add anything that can revert in the swap phase if it can be handled elsewhere. The quote is locked during the reservation process. If you revert during the swap, it would cause a catastrophic loss of Bitcoin, so this must never happen. Partial fills are supported; this is actually a feature we added as a fallback mechanism for swaps. Also, this method cannot be directly called. Any third party wanting to build their own UI to support native swaps must follow all the rules we implemented, or they risk major BTC loss for their users.

3.8. I-02 - Price is updated multiple times in a block [INFO] ACKED

SCOPE

- Commit: 5175d8688e6e3db85324ab92a7427247b0690fbb
- Files:
 - src/managers/LiquidityQueue.ts
 - src/managers/QuoteManager.ts

DESCRIPTION

NativeSwap is designed to update the price at most once per block, but the current code sets the block quote price every time there is a new listing or reservation.

```
// src/operations/ReserveLiquidityOperation.ts#L62-L75
public override execute(): void {
    this.checkPreConditions();
    const reservation: Reservation = this.createReservation();
    this.getValidQuote();
    this.liquidityQueue.purgeReservationsAndRestoreProviders();
    this.ensureEnoughLiquidity();
    this.computeTokenRemaining();
    this.reserve(reservation);
    this.ensureMinimumTokenReserved();
    this.liquidityQueue.increaseTotalReserved(this.reservedTokens);
    this.liquidityQueue.addReservation(reservation);
    this.liquidityQueue.setBlockQuote();
    this.emitReservationCreatedEvent();
}
```

```
// src/operations/ListTokensForSaleOperation.ts#L306-L325
public setBlockQuote(): void {
    this.quoteManager.setBlockQuote(Blockchain.block.number, this.quote());
}
```

```
// src/managers/LiquidityQueue.ts#L419-L421
public setBlockQuote(): void {
    this.quoteManager.setBlockQuote(Blockchain.block.number, this.quote());
}
```

```
// src/managers/QuoteManager.ts#L46-L53
public setBlockQuote(blockNumber: u64, value: u256): void {
    const blockNumberU32: u32 = <u32>(blockNumber % MAXIMUM_QUOTE_INDEX);
    const currentQuote: u256 = this._quoteHistory.get(blockNumberU32);
    if (!u256.eq(currentQuote, value)) {
        this._quoteHistory.set(blockNumberU32, value);
    }
}
```

`setBlockQuote` uses `quote()` , which depends on `virtualTokenReserve` and `virtualSatoshisReserve` . These values are designed to update only once per block, during the first reservation or listing, through `updateVirtualPoolIfNeeded` .

```
// src/managers/LiquidityQueue.ts#L375-L395
public quote(): u256 {
    const TOKEN: u256 = this.virtualTokenReserve;
    const BTC: u64 = this.virtualSatoshisReserve;
    if (TOKEN.isZero()) {
        return u256.Zero;
    }
    if (BTC === 0) {
```

```

    throw new Revert(`Impossible state: Not enough liquidity.`);
  }
  // Calculate queue impact
  const queueImpact = this.calculateQueueImpact();
  // Add impact to token reserves ONLY for price calculation
  const effectiveT = SafeMath.add(TOKEN, queueImpact);
  const scaled = SafeMath.mul(effectiveT, QUOTE_SCALE);
  return SafeMath.div(scaled, u256.fromU64(BTC));
}

```

But `quote()` is also affected by `calculateQueueImpact`, which is derived from the liquidity queue reserves. So within a single block, if there's a listing or a cancellation, the queue impact changes—and so does the quote (could be increased or decreased). Even without `calculateQueueImpact`, `virtualTokenReserve` still increases on listings via `activateSlashing`.

```

// src/managers/LiquidityQueue.ts#L490-L505
private calculateQueueImpact(): u256 {
  const queuedTokens = this.liquidity;
  if (queuedTokens.isZero()) {
    return u256.Zero;
  }
  // Calculate ratio = 1 + Q/T
  const ratio = SafeMath.add(u256.One, SafeMath.div(queuedTokens, this.virtualTokenReserve));
  // Use precise logarithm calculation
  const lnValue = preciseLog(ratio);
  // Impact = T * ln(1 + Q/T) / 1e6 (since log is scaled)
  return SafeMath.div(SafeMath.mul(this.virtualTokenReserve, lnValue), u256.fromU64(1000000));
}

```

After the quote changes, it can be written to the block quote history again, causing the price to update more than once per block (last-write-wins). If someone makes a reservation before that update, then when they call swap they'll receive tokens based on the latest price updated in that block—rather than the price they reserved (`getValidBlockQuote`). This may lead to the user receiving fewer tokens than they reserved.

```

// src/managers/QuoteManager.ts#L33-L38
public getValidBlockQuote(blockNumber: u64): u256 {
  const quote: u256 = this.getBlockQuote(blockNumber);
  this.ensureQuoteIsValid(quote, blockNumber);
  return quote;
}

```

`setBlockQuote` should be called at most once per block to keep the price constant within that block. For unexpired swaps, use the reservation's quote instead of reading from the block-quote history to avoid any price changes (in case there is).

UPDATES

The OP_NET team said that this is a changed behavior from early stage:

The report on C-02 is actually a good feature, and not a bug/exploit. We will modify our documentation to explain it correctly. Having the price change based on sells during block updates from people listing tokens is actually better than not having it, as it allows for better price discovery.

The quoting happen on the reservation. not the swap itself, the user knows what is he about to pay for in the reserve process. He may review his trade before sending the second tx on the swap FE.

4. VERSION HISTORY

Version	Date	Status/Change	Changed by
0.1	<i>Aug 27, 2025</i>	Draft Report	Verichains Lab
0.2	<i>Oct 14, 2025</i>	Draft Report	Verichains Lab
0.3	<i>Nov 21, 2025</i>	Draft Report	Verichains Lab
0.4	<i>Nov 25, 2025</i>	Draft Report	Verichains Lab
1.0	<i>Jan 15, 2026</i>	Private Report	Verichains Lab
1.1	<i>Mar 13, 2026</i>	Public Report	Verichains Lab